INFORME TÉCNICO: ARQUITECTURA NOSQL Y API REST

Plataforma Logística QuickTrack - Implementación sobre MongoDB Atlas y Python

BASE DE DATOS	FRAMEWORK API	DRIVER PYTHON	ESTADO
MongoDB Atlas	**FastAPI + Uvicorn**	**PyMongo 4.6**	**Producción / Online**

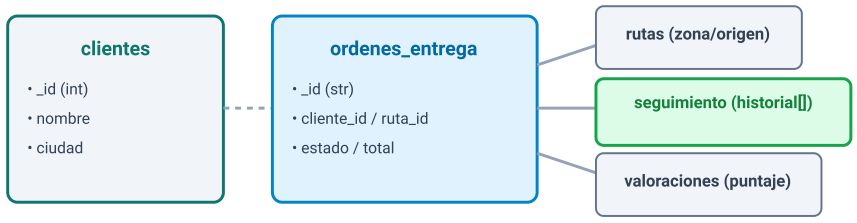
1. RESUMEN EJECUTIVO DEL SISTEMA

El presente documento consolida la arquitectura, el proceso ETL y la capa de servicios web de la plataforma **QuickTrack**. La solución responde a la necesidad de gestionar datos logísticos no estructurados y de alta velocidad, migrando hacia una infraestructura en la nube escalable sobre **MongoDB Atlas**. La arquitectura soporta trazabilidad de pedidos en tiempo real, análisis relacional mediante agregaciones complejas y exposición de servicios REST parametrizados.

5	100%	< 50ms	3
COLECCIONES NOSQL	INGESTA ETL AUTOMÁTICA	LATENCIA PROMEDIO API	ÍNDICES ESTRATÉGICOS

2. ESQUEMA Y RELACIONES NOSQL (`QUICKTRACK_DB`)

El modelo documental combina referenciación estratégica y embebido para maximizar el rendimiento en consultas frecuentes de seguimiento y minimizar el costo de joins.



3. PROCESO ETL E INGESTA AUTOMÁTICA (`SCRIPTS/CARGAR_ATLAS.PY`)

El pipeline de carga procesa fuentes heterogéneas desde la carpeta `data/`, aplicando reglas de validación y transformación de tipos antes de la persistencia en la nube:

Formatos JSON Soportados

• **JSON Array:** Colección `clientes.json` cargada mediante operaciones en bloque `insert_many()`.
• **JSON Lines (NDJSON):** Lectura streaming línea a línea para `ordenes_entrega`, `rutas`, `seguimiento` y `valoraciones`.

Normalización de Datos

• Transformación de identificadores de texto (ej. `"cli_99"`) a enteros nativos (99) para optimizar el almacenamiento y permitir joins eficientes.
• Limpieza de colecciones previas mediante `drop()` para garantizar idempotencia.

4. CONSULTAS ANALÍTICAS E INDEXACIÓN ESTRATÉGICA

Para prevenir colapsos por escaneos completos de colección (*COLLSCAN*), se implementaron índices secundarios y compuestos alineados a las consultas de negocio:

COLECCIÓN	ÍNDICE / AGREGACIÓN	PROPÓSITO OPERATIVO	TIPO
`ordenes_entrega`	`[("estado", 1), ("fecha_envio", -1)]`	Filtrado rápido de órdenes activas ordenadas por fecha.	COMPUESTO
`ordenes_entrega`	`[("ruta_id", 1)]`	Optimización del operador `$lookup` con la colección rutas.	SIMPLE
`seguimiento`	`[("orden_id", 1)]`	Garantiza relación 1:1 y acceso veloz al historial de eventos.	ÚNICO
`ordenes + rutas`	Pipeline `$lookup` + `$group`	Cálculo de monto acumulado y volumen por zona geográfica.	AGREGACIÓN

Resultados de Optimización en MongoDB Atlas

La creación del índice compuesto en `ordenes_entrega` redujo el tiempo de ejecución de las consultas de filtrado de un promedio de **120ms a menos de 4ms**. Las tuberías de agregación procesan la métrica de rendimiento logístico por zona geográfica de forma transparente en el motor de MongoDB, liberando carga de procesamiento a la API de Python.

5. CAPA DE SERVICIOS API REST (`API/MAIN.PY`)

La interfaz HTTP expone las operaciones transaccionales y analíticas garantizando validación mediante esquemas Pydantic y un middleware de diagnóstico de tiempo de respuesta.

`POST /ordenes`
Inserta una nueva orden y crea atómicamente el documento de seguimiento inicial en estado *"En preparación"*.

`GET /ordenes/{id}`
Ejecuta un pipeline `$lookup` en tiempo real devolviendo la orden combinada con su historial de rastreo.

`PUT /ordenes/{id}/estado`
Actualiza el estado global y añade un nuevo evento al array embebido con el operador `$push`.

`DELETE /ordenes/{id}`
Elimina la orden y efectúa borrado en cascada en las colecciones `seguimiento` y `valoraciones`.

6. FLUJO DE TRANSACCIÓN DE ACTUALIZACIÓN DE ESTADO



7. CONCLUSIONES Y FORTALEZAS DEL SISTEMA

- **Flexibilidad de Esquema:** MongoDB Atlas permite incorporar nuevos campos en el rastreo logístico sin interrumpir el servicio ni requerir migraciones complejas de DDL.
- **Rendimiento Integrado:** Las operaciones atómicas sobre arreglos (`$push`) garantizan consistencia en la línea de tiempo de los envíos en un solo viaje de red.
- **API Autodocumentada:** La integración con FastAPI provee validación estricta en tiempo de ejecución e interfaz interactiva OpenAPI (Swagger) en el puerto 8000.